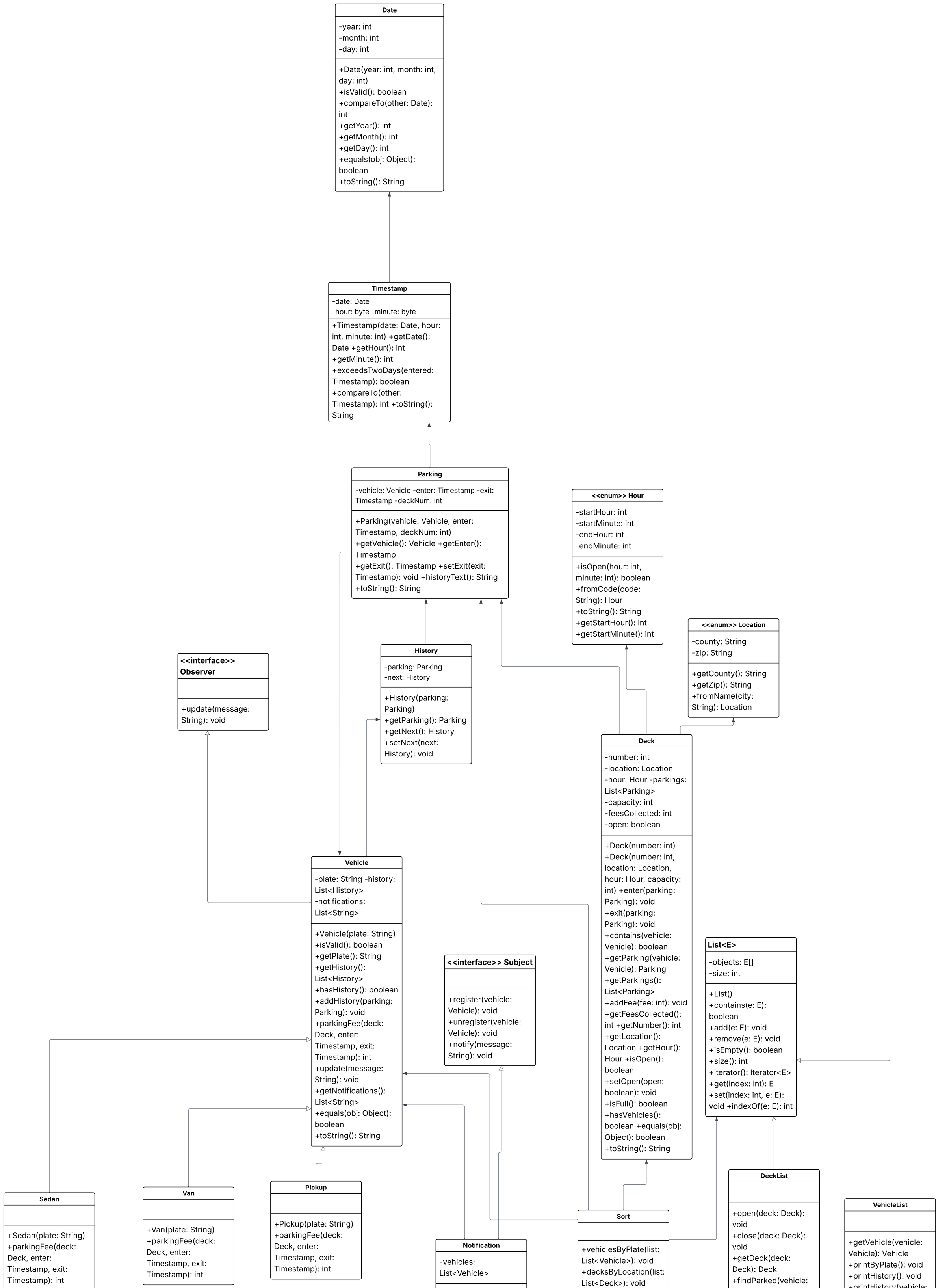

Date
-year: int
-month: int
-day: int
+Date(year: int, month: int, day: int)
+isValid(): boolean
+compareTo(other: Date): int
+getYear(): int
+getMonth(): int
+getDay(): int
+equals(obj: Object): boolean
+toString(): String
Timestamp
-date: Date
-hour: byte -minute: byte
+Timestamp(date: Date, hour: int, minute: int) +getDate(): Date +getHour(): int
+getMinute(): int
+exceedsTwoDays(entered: Timestamp): boolean
+compareTo(other: Timestamp): int +toString(): String
Parking
-vehicle: Vehicle -enter: Timestamp -exit: Timestamp -deckNum: int
+Parking(vehicle: Vehicle, enter: Timestamp, deckNum: int)
+getVehicle(): Vehicle +getEnter(): Timestamp
+getExit(): Timestamp +setExit(exit: Timestamp): void +historyText(): String
+toString(): String
<<enum>> Hour
-startHour: int
-startMinute: int
-endHour: int
-endMinute: int
+isOpen(hour: int, minute: int): boolean
+fromCode(code: String): Hour
+toString(): String
+getStartHour(): int
+getStartMinute(): int
<<enum>> Location
-county: String
-zip: String
+getCounty(): String
+getZip(): String
+fromName(city: String): Location
<<interface>> Observer
+update(message: String): void
History
-parking: Parking
-next: History
+History(parking: Parking)
+getParking(): Parking
+getNext(): History
+setNext(next: History): void
Deck
-number: int
-location: Location
-hour: Hour -parkings: List<Parking>
-capacity: int
-feesCollected: int
-open: boolean
+Deck(number: int)
+Deck(number: int, location: Location, hour: Hour, capacity: int) +enter(parking: Parking): void
+exit(parking: Parking): void
+contains(vehicle: Vehicle): boolean
+getParking(vehicle: Vehicle): Parking
+getParkings(): List<Parking>
+addFee(fee: int): void
+getFeesCollected(): int +getNumber(): int
+getLocation(): Location +getHour(): Hour +isOpen(): boolean
+setOpen(open: boolean): void
+isFull(): boolean
+hasVehicles(): boolean +equals(obj: Object): boolean
+toString(): String
Vehicle
-plate: String -history: List<History>
-notifications: List<String>
+Vehicle(plate: String)
+isValid(): boolean
+getPlate(): String
+getHistory(): List<History>
+hasHistory(): boolean
+addHistory(parking: Parking): void
+parkingFee(deck: Deck, enter: Timestamp, exit: Timestamp): int
+update(message: String): void
+getNotifications(): List<String>
+equals(obj: Object): boolean
+toString(): String
<<interface>> Subject
+register(vehicle: Vehicle): void
+unregister(vehicle: Vehicle): void
+notify(message: String): void
List<E>
-objects: E[]
-size: int
+List()
+contains(e: E): boolean
+add(e: E): void
+remove(e: E): void
+isEmpty(): boolean
+size(): int
+iterator(): Iterator<E>
+get(index: int): E
+set(index: int, e: E): void +indexOf(e: E): int
Sedan
+Sedan(plate: String)
+parkingFee(deck: Deck, enter: Timestamp, exit: Timestamp): int
Van
+Van(plate: String)
+parkingFee(deck: Deck, enter: Timestamp, exit: Timestamp): int
Pickup
+Pickup(plate: String)
+parkingFee(deck: Deck, enter: Timestamp, exit: Timestamp): int
Notification
-vehicles: List<Vehicle>
+Notification()
+register(vehicle: Vehicle): void
+unregister(vehicle: Vehicle): void
+notify(message: String): void
Sort
+vehiclesByPlate(list: List<Vehicle>): void
+decksByLocation(list: List<Deck>): void
+decksByNumber(list: List<Deck>): void
+parkingsByPlate(list: List<Parking>): void
DeckList
+open(deck: Deck): void
+close(deck: Deck): void
+getDeck(deck: Deck): Deck
+findParked(vehicle: Vehicle): Deck
+printByLocation(): void
+printVehicles(deck: Deck): void
+printFees(): void
VehicleList
+getVehicle(vehicle: Vehicle): Vehicle
+printByPlate(): void
+printHistory(): void
+printHistory(vehicle: Vehicle): void
+printNotifications(): void
+printNotifications(vehicle: Vehicle): void
Operation
-vehicles: VehicleList
-decks: DeckList
-notification: Notification
+Operation()
+run(): void
RunProject2
+main(args: String[]): void